

Chapter 3

Autoencoder Asset Pricing

Giovanni Della Lunga
giovanni.dellalunga@unibo.it

Advanced Topics in Artificial Intelligence

Bologna - March, 2026

1. The Conditional Autoencoder

- The Big Picture
- Beta network in detail
- Factor network in detail
- Dot Product and Full Model
- Conclusions and Summary

2. Building the Dataset

- Data Download and Pre-Processing

3. Data Preparation and Feature Engineering

- Data Loading & Alignment
- Factor Engineering
- Storage & Output
- Summary of Part 3

The Conditional Autoencoder

Section 1.1

The Big Picture

Recap: Where We Left Off

From Lecture 2:

- **Part 1:** Static factor models $r_t = \beta f_t + u_t$; PCA extracts latent factors but β is constant.
- **Part 2:** Linear autoencoder = PCA (Proposition 1). Adding nonlinearity generalizes PCA.
- **Part 3:** IPCA makes β depend on characteristics: $\beta(z_{i,t-1})' = z_{i,t-1}'\Gamma$ (linear). But linearity is restrictive – theory predicts nonlinear relationships.

Today: We enter the **core** of the paper – the *Conditional Autoencoder*, which:

- ① Models β as a **nonlinear** function of characteristics (neural network)
- ② Estimates **latent** factors as portfolios (linear combinations of returns)
- ③ Combines both via a **dot product** to predict returns
- ④ Imposes the economic restriction of **no-arbitrage**

The Model at the Highest Level (Eq. 9)

The conditional autoencoder is, at the highest level, still a factor model:

Equation (9) of the paper

$$r_{i,t} = \beta'_{i,t-1} f_t + u_{i,t}$$

Same structure as IPCA – but the key differences are:

	IPCA	Conditional Autoencoder
$\beta_{i,t-1}$	$\Gamma' z_{i,t-1}$ (linear)	$\text{NN}(z_{i,t-1}; \theta)$ (nonlinear)
f_t	Latent (from FOCs)	Linear combination of returns
Interactions	None (additive)	Learned by hidden layers

The model has **two sub-networks** that are trained *jointly* end-to-end.

Bird's Eye View: Two Sub-Networks

LEFT SIDE

Characteristics
 $z_{i,t-1}$
($P \times 1$)

Beta Network
(neural network)
Hidden layers + ReLU

$\beta_{i,t-1}$
($K \times 1$)

Dot Product
 $\hat{r}_{i,t} = \beta'_{i,t-1} f_t$

$\hat{r}_{i,t}$
(scalar)

Managed portfolios
 x_t
($P \times 1$)

Factor Network
(single linear layer)

f_t
($K \times 1$)

RIGHT SIDE

What Each Network Does

Beta Network (left side)

Input: P firm characteristics $z_{i,t-1}$

Output: K factor loadings $\beta_{i,t-1}$

Architecture: Deep neural network with ReLU activation and hidden layers

Key feature: Captures *nonlinear* and *interactive* effects of characteristics on risk exposures

One network, applied to each stock i
(weights shared across stocks)

Factor Network (right side)

Input: P managed portfolios x_t (or N returns r_t)

Output: K latent factor returns f_t

Architecture: Single *linear* layer (no hidden layers, no activation)

Key feature: Factors remain *portfolios* (linear combinations of returns) – economically interpretable

One network for the whole cross section

Section 1.2

Beta network in detail

The Beta Network: Mathematical Formulation

The beta network transforms characteristics into factor loadings through L_β hidden layers:

Equations (10)–(12) of the paper

$$z_{i,t-1}^{(0)} = z_{i,t-1} \quad (10)$$

$$z_{i,t-1}^{(l)} = g\left(b^{(l-1)} + W^{(l-1)} z_{i,t-1}^{(l-1)}\right), \quad l = 1, \dots, L_\beta \quad (11)$$

$$\beta_{i,t-1} = b^{(L_\beta)} + W^{(L_\beta)} z_{i,t-1}^{(L_\beta)} \quad (12)$$

Eq. (10): Initialize with raw characteristics (input layer).

Eq. (11): Apply nonlinear transformation (ReLU) at each hidden layer.

Eq. (12): Output layer is **linear** (no activation) – produces K beta values.

Equation (10): Initialization

$$z_{i,t-1}^{(0)} = z_{i,t-1} \in \mathbb{R}^{P \times 1}$$

What is $z_{i,t-1}$?

- A vector of P observable characteristics (94 in the original paper) for stock i at time $t - 1$
- Includes: momentum, size, value, volatility, liquidity, profitability, investment, ...
- All characteristics are **rank-normalized** into $(-1, 1)$ each month
- Missing values replaced by the cross-sectional median
- Characteristics are **lagged**: we use $z_{i,t-1}$ to predict returns at time t

Why normalize?

- Raw characteristics have very different scales (market cap in billions vs. beta ≈ 1)
- Rank normalization ensures all inputs are comparable and bounded
- Prevents outliers from dominating the network
- Makes optimization more stable

Equation (11): Hidden Layer Propagation

For each hidden layer $l = 1, \dots, L_\beta$:

$$z_{i,t-1}^{(l)} = g\left(b^{(l-1)} + W^{(l-1)}z_{i,t-1}^{(l-1)}\right)$$

Components:

- $W^{(l-1)} \in \mathbb{R}^{K^{(l)} \times K^{(l-1)}}$: weight matrix connecting layer $l - 1$ to layer l
- $b^{(l-1)} \in \mathbb{R}^{K^{(l)}}$: bias vector for layer l
- $g(\cdot) = \max(\cdot, 0)$: ReLU activation (applied element-wise)
- $K^{(l)}$: number of neurons in layer l

What happens at each layer:

- 1 **Linear transformation:** $W^{(l-1)}z_{i,t-1}^{(l-1)} + b^{(l-1)}$ – mixes and recombines features
- 2 **Nonlinear activation:** ReLU sets negative values to zero – creates *sparsity* and *piecewise linearity*

Crucial: The **same weights** $W^{(l)}, b^{(l)}$ are used for every stock i . The only thing that changes across stocks is the input $z_{i,t-1}$.

Where Nonlinearity and Interactions Come From

Example: Two characteristics z_1 (momentum) and z_2 (size).

After one hidden layer with 2 neurons:

$$h_1 = \max(0, w_{11}z_1 + w_{12}z_2 + b_1), \quad h_2 = \max(0, w_{21}z_1 + w_{22}z_2 + b_2)$$

Output: $\beta_k = v_1 h_1 + v_2 h_2 + b_k^{\text{out}}$

This captures:

- **Nonlinear effects:** β changes slope at different levels of z_1 (due to ReLU kink)
- **Interactions:** h_1 mixes z_1 and z_2 together, so β depends on their *combination*
- **Threshold effects:** If $w_{11}z_1 + w_{12}z_2 + b_1 < 0$, neuron h_1 is off – β depends only on h_2

Compare with IPCA

IPCA: $\beta_k = \gamma_{k1}z_1 + \gamma_{k2}z_2$ (purely additive, no interactions, no thresholds). The neural network is a **strict generalization**.

Equation (12): The Output Layer

$$\beta_{i,t-1} = b^{(L_\beta)} + W^{(L_\beta)} z_{i,t-1}^{(L_\beta)} \in \mathbb{R}^{K \times 1}$$

The output layer is linear (no ReLU). Why?

- Factor loadings $\beta_{i,k}$ can be positive or negative
- ReLU would clip negative betas to zero – destroying important information
- We want betas to be *unconstrained* real numbers

Output dimensionality:

- $W^{(L_\beta)} \in \mathbb{R}^{K \times K^{(L_\beta)}}$: maps from last hidden layer to K outputs
- Each output neuron represents one **factor loading** $\beta_{i,k}$ for $k = 1, \dots, K$
- So the full output is $\beta_{i,t-1} = (\beta_{i,1,t-1}, \dots, \beta_{i,K,t-1})'$

For each stock i at each time t : the network produces a *personalized* vector of K risk exposures, determined by that stock's current characteristics.

Weight Sharing Across Stocks

Critical design feature: The beta network has **shared weights**.

The same parameters $\{W^{(l)}, b^{(l)}\}_{l=0}^{L_\beta}$ are used for *all* stocks $i = 1, \dots, N$.

What changes across stocks: Only the input $z_{i,t-1}$.

Why this matters:

- Without weight sharing: would need $\sim 30,000 \times$ (parameters per stock) – infeasible
- With weight sharing: the network learns a *universal mapping* from characteristics to betas
- A new stock (IPO) gets immediate beta estimates – just feed its characteristics through the trained network
- Stocks with similar characteristics get similar betas – a form of *regularization*

Analogy: Think of the beta network as a “beta calculator” – you input a stock’s characteristics, and it outputs that stock’s risk profile. The calculator is the same for everyone; the results differ because the inputs differ.

Section 1.3

Factor network in detail

The Factor Network: Overview

The right side of Figure 2 estimates the latent factors f_t .

Ideally, we would use the full cross section of returns $r_t \in \mathbb{R}^N$:

$$r_t^{(0)} = r_t \tag{13}$$

$$r_t^{(l)} = g\left(\tilde{b}^{(l-1)} + \tilde{W}^{(l-1)} r_t^{(l-1)}\right), \quad l = 1, \dots, L_f \tag{14}$$

$$f_t = \tilde{b}^{(L_f)} + \tilde{W}^{(L_f)} r_t^{(L_f)} \tag{15}$$

But there's a big problem...

The Dimensionality Problem

Why we can't use r_t directly:

Problem 1: Too many parameters.

- With $N \approx 30,000$ stocks: the first layer would need $K \times 30,000$ weight parameters
- With $T = 720$ months: massively more parameters than observations
- Guaranteed to overfit catastrophically

Problem 2: Unbalanced panel.

- On average $\sim 6,200$ stocks per month, but different stocks each month
- Total universe: $\sim 30,000$ stocks (enter/exit due to IPOs, delistings, mergers)
- Most stock-level weights would be estimated from very few time-series observations

We need a way to reduce dimensionality *before* entering the network.

The Solution: Managed Portfolios (Eq. 16)

Key insight: Replace the N -dimensional return vector with P -dimensional managed portfolios.

Equation (16) of the paper

$$x_t = (Z'_{t-1} Z_{t-1})^{-1} Z'_{t-1} r_t$$

Dimensions:

- $Z_{t-1} \in \mathbb{R}^{N_t \times P}$: matrix of characteristics for all N_t stocks at time $t - 1$
- $r_t \in \mathbb{R}^{N_t}$: returns of all N_t stocks at time t
- $x_t \in \mathbb{R}^P$: vector of $P = 94$ managed portfolio returns (plus one market portfolio)

This is a **cross-sectional regression** of returns on characteristics at each time t :

$$x_t = \arg \min_x \|r_t - Z_{t-1} x\|^2$$

The j -th element of x_t is the return on a long-short portfolio sorted by characteristic j .

Three Benefits of Using Managed Portfolios

Benefit 1: Massive dimensionality reduction.

- From $N \approx 30,000$ individual returns to $P = 95$ portfolio returns
- Eliminates tens of thousands of parameters from the factor network

Benefit 2: Handles panel incompleteness.

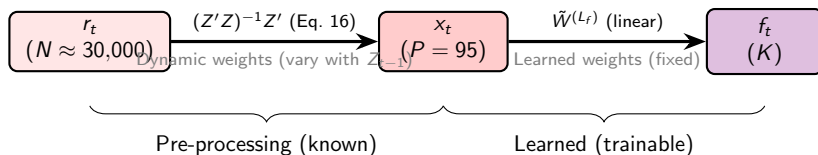
- Portfolios are formed from whichever stocks are available at each point in time
- No need for a balanced panel – new stocks and delistings are handled automatically

Benefit 3: Connection to the finance literature.

- Characteristic-managed portfolios are well-studied (KPS, Feng et al., Kozak et al.)
- KPS and Giglio & Xiu show that conditional linear models for individual stocks can be recast as static factor analysis on managed portfolios
- Provides a natural bridge between individual stock models and portfolio-level analysis

Managed Portfolios as a Pre-Processing Layer

Conceptual view: x_t adds a new initial layer to the factor network.



The pre-processing step dynamically reweights stocks based on their characteristics – it's a data-driven dimensionality reduction that varies over time.

The Factor Network: Final Specification

With managed portfolios as input, the factor network becomes very simple:

Single linear layer ($L_f = 1$, no hidden layers, no activation):

$$f_t = \tilde{b}^{(L_f)} + \tilde{W}^{(L_f)} x_t$$

Dimensions: $\tilde{W}^{(L_f)} \in \mathbb{R}^{K \times P}$, $\tilde{b}^{(L_f)} \in \mathbb{R}^K$.

Why keep the factor network linear?

Economic interpretability

- $f_t = \tilde{W} x_t + \tilde{b}$: each factor is a **linear combination of portfolio returns**
- Linear combinations of returns are themselves **portfolios**
- So factors are tradeable – they have direct economic meaning
- A nonlinear factor network would produce factors that are *not* portfolios, losing interpretability

Section 1.4

Dot Product and Full Model

The Dot Product: Combining Both Networks

The final prediction for stock i at time t :

Output

$$\hat{r}_{i,t} = \beta'_{i,t-1} f_t = \sum_{k=1}^K \beta_{i,k,t-1} \cdot f_{k,t}$$

This is a **dot product** (inner product) between:

- $\beta_{i,t-1} \in \mathbb{R}^K$: from the beta network (stock-specific, time-varying)
- $f_t \in \mathbb{R}^K$: from the factor network (common to all stocks, time-varying)

For the full cross section at time t :

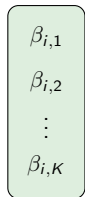
$$\hat{r}_t = B_{t-1} f_t \in \mathbb{R}^N$$

where $B_{t-1} = (\beta_{1,t-1}, \dots, \beta_{N,t-1})' \in \mathbb{R}^{N \times K}$ is the matrix of all betas.

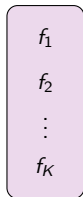
This is exactly a factor model: $\hat{r}_t = \beta_t f_t$, but with **time-varying, nonlinear** betas.

The Dot Product as a Bilinear Operation

From beta net



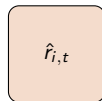
From factor net



.

=

Predicted return



$$= \beta_{i,1}f_1 + \beta_{i,2}f_2 + \cdots + \beta_{i,K}f_K$$

Stock i 's return = sum of
(its exposure to each factor)
× (that factor's realization)

Key: The dot product enforces the **factor model structure**. Without it, the network would be a generic prediction model (like GKX, 2019) with no economic structure.

Complete Forward Pass: Step by Step

For stock i at time t , the conditional autoencoder computes:

- 1 **Read characteristics:** $z_{i,t-1} \in \mathbb{R}^P$ (lagged, rank-normalized)
- 2 **Beta network:** $z_{i,t-1} \xrightarrow{W^{(0)}, \text{ReLU}} h_1 \xrightarrow{W^{(1)}, \text{ReLU}} h_2 \xrightarrow{W^{(2)}, \text{linear}} \beta_{i,t-1} \in \mathbb{R}^K$
- 3 **Compute managed portfolios:** $x_t = (Z'_{t-1} Z_{t-1})^{-1} Z'_{t-1} r_t \in \mathbb{R}^{95}$
(pre-processing, done once for all stocks at time t)
- 4 **Factor network:** $x_t \xrightarrow{\tilde{W}, \text{linear}} f_t \in \mathbb{R}^K$
- 5 **Dot product:** $\hat{r}_{i,t} = \beta'_{i,t-1} f_t \in \mathbb{R}$

Repeat for all N_t stocks at time t , then for all T months.

The Role of Each Component: Summary Table

Component	Input	Output	Key property
Beta network	$z_{i,t-1} \in \mathbb{R}^{94}$ (characteristics)	$\beta_{i,t-1} \in \mathbb{R}^K$ (loadings)	Nonlinear, shared weights
Factor network	$x_t \in \mathbb{R}^{95}$ (managed portfolios)	$f_t \in \mathbb{R}^K$ (factors)	Linear, factors = portfolios
Dot product	$\beta_{i,t-1}, f_t$	$\hat{r}_{i,t}$ (scalar)	Enforces factor structure
Managed port.	r_t, Z_{t-1}	x_t	Pre-processing, not learned

Why This Architecture?

The design of the conditional autoencoder reflects a careful balance of **economic structure** and **statistical flexibility**:

- **Factor model structure** ($r = \beta' f + u$): preserved via the dot product. This is not a black-box predictor – it's an asset pricing model.
- **Nonlinear betas**: The neural network allows flexible, data-driven risk exposures that capture interactions and thresholds.
- **Linear factors**: Keeping factors as portfolios maintains economic interpretability and connects to the finance literature.
- **Managed portfolios**: Solve the dimensionality and unbalanced panel problems elegantly.
- **Weight sharing**: Makes the model feasible for $\sim 30,000$ stocks and provides implicit regularization.
- **No intercept**: Imposes the no-arbitrage restriction (more in Part B).

What Makes This a “Conditional” Autoencoder?

Standard autoencoder (Lecture 2):

- Input: returns r_t
- Hidden: factors (bottleneck)
- Output: reconstructed returns $\hat{r}_t$
- **Unsupervised:** uses only returns

Conditional autoencoder (this paper):

- **Conditions** the loadings on external information $z_{i,t-1}$
- Returns enter only through managed portfolios on the factor side
- Characteristics enter only on the beta side
- **Semi-supervised:** uses returns *and* characteristics

The conditioning is what allows betas to be **time-varying** and **stock-specific**: two stocks with different characteristics at the same time will have different betas, and the same stock at different times (with changed characteristics) will have different betas.

Section 1.5

Conclusions and Summary

Part A – Summary

- 1 The conditional autoencoder maintains the factor model structure $r_{i,t} = \beta'_{i,t-1} f_t + u_{i,t}$, but with two neural sub-networks.
- 2 The **beta network** (Eqs. 10–12) maps 94 characteristics to K loadings through hidden layers with ReLU activation. Architectures: CA0 (linear), CA1 (32), CA2 (32-16), CA3 (32-16-8).
- 3 The **factor network** (Eqs. 13–16) uses managed portfolios x_t as input and a single linear layer, keeping factors interpretable as portfolios.
- 4 Managed portfolios (Eq. 16) solve the dimensionality and unbalanced panel problems by reducing $N \approx 30,000$ returns to $P = 95$ portfolio returns.
- 5 The **dot product** $\hat{r}_{i,t} = \beta'_{i,t-1} f_t$ combines both networks and enforces the factor model structure.
- 6 Weight sharing across stocks keeps the model tractable ($\sim 4,000$ parameters).

Next (Part B): Equivalence with IPCA (Proposition 2) and the no-arbitrage restriction.

Building the Dataset

Section 2.1

Data Download and Pre-Processing

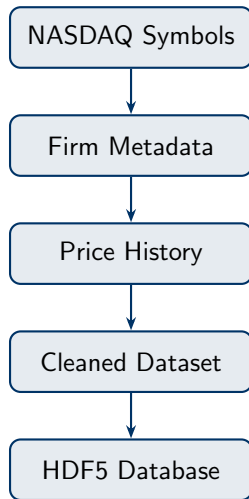
Why Build a Custom Equity Dataset?

Research Needs

- Empirical asset pricing requires **large cross-sections** of stock returns
- Machine learning applications need **clean, structured data**
- Commercial databases (CRSP, Compustat) are expensive

Notebook Goals

- Build a **free, reproducible** U.S. equity dataset
- Combine **NASDAQ symbol directory** + **Yahoo Finance**
- Store results in an efficient **HDF5** database



What the Dataset Contains

[M] Equity Metadata

- Company name, sector, industry
- Market capitalisation
- Descriptive attributes from Yahoo Finance
- One record per ticker

[P] Adjusted Price Series

- Open, High, Low, Close, Volume
- Adjusted for splits and dividends
- Multi-index: (ticker, date)
- Long historical horizon

[D] Storage Format — HDF5

Hierarchical Data Format allows fast I/O for large numerical arrays.

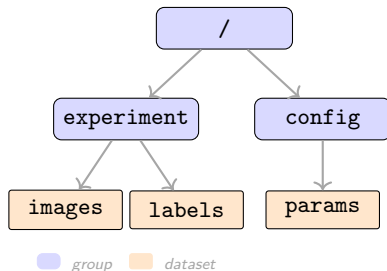
Keys: `stocks/info` (metadata) and `stocks/prices/adjusted` (prices).

What is HDF5?

HDF5 (Hierarchical Data Format v5) is an open-source binary file format designed for storing and organizing **large, complex datasets**.

Core concepts

- **Groups** — folder-like containers that form a tree hierarchy (like a filesystem inside a file)
- **Datasets** — typed, multidimensional arrays of any size
- **Attributes** — lightweight key-value metadata attached to groups or datasets



Why Use HDF5?

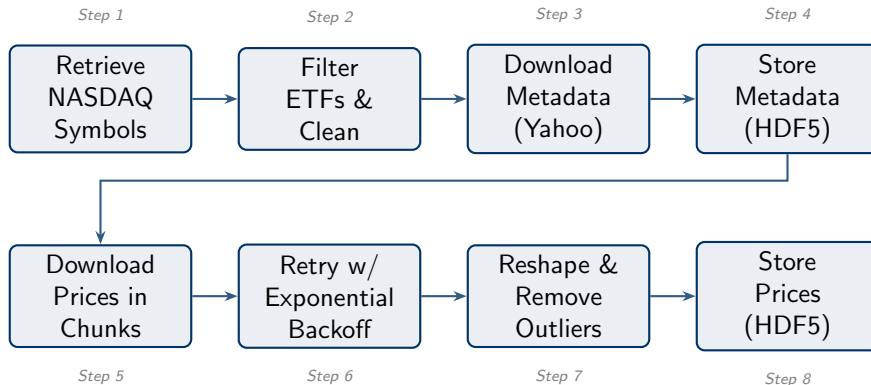
Key advantages

- ① **Scalability** — handles datasets from kilobytes to terabytes in a single file
- ② **Fast I/O** — supports chunked storage and built-in compression (gzip, lz4, ...); enables partial reads without loading the whole file
- ③ **Self-describing** — metadata travels with the data, improving reproducibility
- ④ **Portable** — platform-independent binary format; one file works on Linux, macOS, Windows
- ⑤ **Broad ecosystem** — native support in Python (h5py, pandas), MATLAB, R, C/C++, Julia, ...

HDF5 vs. alternatives

Feature	HDF5	CSV	NPY
N-dim arrays	✓	—	✓
Compression	✓	—	—
Partial read	✓	—	—
Metadata	✓	—	—
Multi-dataset	✓	—	—
Human-readable	—	✓	—

End-to-End Pipeline



Retrieving NASDAQ Symbols

The notebook uses `pandas_datareader` to obtain the full NASDAQ symbol directory.

```
from pandas_datareader.nasdaq_trader import get_nasdaq_symbols  
  
traded_symbols = get_nasdaq_symbols()
```

Filtering Logic

- ❶ **Exclude ETFs:** retain only individual equities (`~traded_symbols.ETF`)
- ❷ **Drop missing values:** remove NaN tickers
- ❸ **Cast to string & strip:** ensure ticker format consistency
- ❹ **Deduplicate:** call `.unique()` to avoid repeated symbols

Result: a clean Python list `all_symbols` containing thousands of equity tickers.

Downloading Firm-Level Metadata

For each ticker, the notebook queries Yahoo Finance via `yfinance`:

```
yf_symbols = yf.Tickers(all_symbols)
for ticker, yf_object in tqdm(yf_symbols.tickers.items()):
    info = yf_object.get_info()          # dict of attributes
    meta_data.append(pd.Series(info).to_frame(ticker))
```

Robustness Features

- **try/except:** failures are logged, not fatal
- **Logging to file:**
`yfinance_download_errors.log`
- **stderr redirect:** suppresses noisy library output
- **Progress bar:** via `tqdm`

Output

- A wide DataFrame: one row per ticker, many attribute columns
- Numeric conversion applied automatically
- Stored in HDF5 under key `stocks/info`

Downloading Prices — Chunking & Retry Strategy

```
MAX_RETRIES = 5
BASE_BACKOFF = 1.5
MIN_INTERVAL = 1.0
for i, chunk in enumerate(chunks(all_symbols, 100), 1):
    for attempt in range(1, MAX_RETRIES + 1):
        try:
            df = yf.download(chunk, period='max',
                              auto_adjust=True, threads=True)
            prices_adj.append(df.stack(-1))
            break
        except Exception as e:
            tm.sleep(BASE_BACKOFF ** attempt) # exponential backoff
```

Three Resilience Mechanisms

Chunking Tickers are split into batches of 100 to avoid oversized API calls.

Retry + Backoff Each failed chunk is retried up to 5 times; wait grows as 1.5^{attempt} .

Rate Pacing A 1-second pause between batches reduces rate-limit risk.

Reshaping the Price DataFrame

After downloading, the raw data is restructured:

```
prices_adj = (pd.concat(prices_adj)
               .dropna(how='all', axis=1)
               .rename(columns=str.lower)
               .swaplevel())
prices_adj.index.names = ['ticker', 'date']
```

Index Structure

A **Multindex** (ticker, date) enables:

- Cross-sectional slicing (all stocks on one date)
- Time-series slicing (one stock over time)
- Efficient group-by operations

Columns (lowercase)

- open, high, low, close
- volume
- All prices are **split- and dividend-adjusted**

Outlier Removal

Extreme return observations are likely data errors. The notebook uses a simple but effective filter:

```
df = prices_adj.close.unstack('ticker')
pmax = df.pct_change().max()
pmin = df.pct_change().min()
to_drop = pmax[pmax > 1].index.union(pmin[pmin < -1].index)
prices_adj = prices_adj.drop(to_drop, level='ticker')
```

Filtering Rule

- Compute daily returns: $r_t = (P_t - P_{t-1})/P_{t-1}$
- If **any** daily return exceeds +100% or falls below -100%, the **entire ticker** is removed
- This guards against price errors that could distort empirical results

Final Storage

The cleaned dataset is written to HDF5 with a date filter:

```
prices_adj.sort_index() \
    .loc[idx[:, '1990':'2019'], :] \
    .to_hdf(results_path / 'data_ip_2026.h5',
            'stocks/prices/adjusted')
```

HDF5 Database Layout

Key	Content	Shape
stocks/info	Firm metadata	tickers $\times$ attributes
stocks/prices/adjusted	Adj. prices 1990–2019	(ticker, date) $\times$ OHLCV

This consolidated file can be loaded efficiently by downstream research notebooks.

Summary of Part 2 & Downstream Uses

What the Notebook Produces

A single HDF5 file containing **firm metadata** and **adjusted prices** for thousands of U.S. equities spanning roughly three decades.

Possible Applications

- **Factor modeling:** Fama–French, momentum, quality factors
- **Portfolio construction:** mean–variance, risk parity, Black–Litterman
- **Machine learning:** return prediction, regime detection, NLP on firm metadata
- **Market microstructure:** liquidity analysis, volatility estimation

Data Preparation and Feature Engineering

Context: Conditional Autoencoders for Asset Pricing

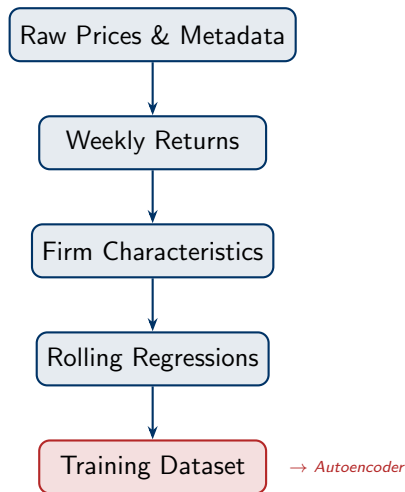
The Bigger Picture

Conditional autoencoders learn **latent factor structures** from asset returns, where factor loadings depend on observable firm characteristics.

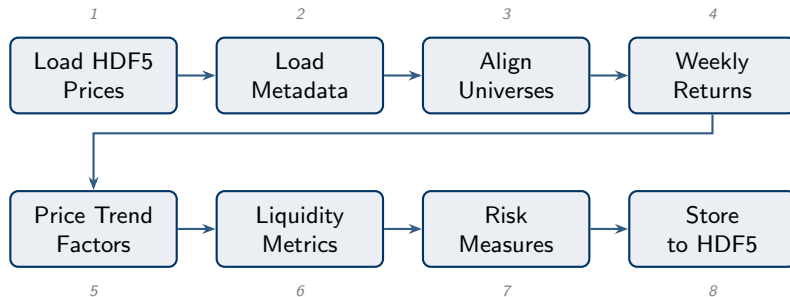
This notebook is **Part 1** of the project: it builds the **input dataset** that the neural network will consume.

Why Data Preparation Matters

Model performance critically depends on **clean returns** and **well-constructed features** — “garbage in, garbage out.”



Notebook Workflow at a Glance



Data Loading

Prices, metadata, universe alignment

Feature Engineering

14 firm-level characteristics

Storage

HDF5 with hierarchical keys

Section 3.1

Data Loading & Alignment

Loading Prices & Metadata from HDF5

```
prices = pd.read_hdf(results_path / 'data_ip_2026.h5',  
                    'stocks/prices/adjusted')  
metadata = pd.read_hdf(results_path / 'data_ip_2026.h5',  
                      'stocks/info').rename(columns=str.lower)
```

Prices

- MultiIndex: (ticker, date)
- Columns: open, high, low, close, volume
- Split- and dividend-adjusted

Metadata

- Index: ticker
- Key columns retained: sector, marketcap, sharesoutstanding
- Sourced from Yahoo Finance

Filtering & Aligning the Ticker Universe

Only tickers that pass **all** quality filters are retained:

```
tickers_with_metadata = metadata[
    metadata.sector.isin(sectors) &          # well-populated sectors (>50 firms)
    metadata.marketcap.notnull() &          # non-missing market cap
    metadata.sharesoutstanding.notnull() &  # non-missing shares
    (metadata.sharesoutstanding > 0)        # positive shares
].index
valid_tickers = tickers_with_metadata.intersection(
    prices.index.get_level_values(0).unique())
prices = prices.loc[idx[valid_tickers, :], :]
```

Alignment Logic

- 1 Filter metadata: sector > 50 firms, valid marketcap and sharesoutstanding
- 2 Intersect with tickers actually present in the price data
- 3 Keep only the **common universe** in both datasets

This guarantees every observation has both prices *and* fundamentals.

Constructing Weekly Returns

```
returns = (prices.close
            .unstack('ticker')           # wide format: dates x tickers
            .resample('W-FRI').last()    # last price each Friday
            .sort_index()
            .pct_change()                # week-over-week returns
            .iloc[1:])                  # drop first NaN row
```

Why Weekly?

- Reduces daily microstructure noise
- Lowers dimensionality vs. daily data
- Better balance between resolution and statistical robustness

Design Choices

- **Friday-to-Friday** alignment ensures consistency
- **Adjusted prices** $\Rightarrow$ returns reflect total shareholder performance
- Result stored in HDF5 under `returns`

Section 3.2

Factor Engineering

Overview of Engineered Features

The notebook constructs **14 firm-level characteristics** organised into three categories:

Price Trend (6)

<code>mom1m</code>	Short-term reversal
<code>mom12m</code>	12-month momentum
<code>chmom</code>	Momentum change
<code>indmom</code>	Industry momentum
<code>maxret</code>	Recent max return
<code>mom36m</code>	Long-term reversal

Liquidity (5)

<code>turn</code>	Turnover
<code>turn_std</code>	Turnover volatility
<code>mvel</code>	Log market equity
<code>dolvol</code>	Dollar volume
<code>ill</code>	Amihud illiquidity

Risk (3)

<code>retvol</code>	Return volatility
<code>beta</code>	Market beta
<code>betasq</code>	Beta squared
<code>idiovol</code>	Idiosyncratic vol.

All factors are computed at **weekly (Friday)** frequency, aligned with the return panel, and stored in long format (date, ticker) in HDF5.

Price Trend Factors — Definitions

Let $P_{i,t}$ be the adjusted closing price of stock i at trading day t , and $M \approx 21$ trading days ≈ 1 month.

Factor	Formula	Interpretation
mom1m	$\frac{P_{i,t}}{P_{i,t-M}} - 1$	Short-term reversal (1-month return)
mom12m	$\frac{P_{i,t-M}}{P_{i,t-12M}} - 1$	Momentum: 11-month return, skip last month
chmom	$R_{t-6:t-1} - R_{t-12:t-7}$	Momentum acceleration / deceleration
indmom	$\frac{1}{N_{s,t}} \sum_{i \in s} r_{i,t}^{(12m)}$	Equal-weighted sector 12-month return
maxret	$\max_k \frac{P_{i,k}}{P_{i,k-M}} - 1$	Max 21-day return in prior month
mom36m	$\frac{P_{i,t-12M}}{P_{i,t-36M}} - 1$	Long-term reversal (months $t-36$ to $t-13$)

Price Trend — Code Pattern

All price-trend factors follow the same pandas chain:

Example: 12-month momentum (skip last month)

```
mom12m = (close
          .pct_change(periods=11 * MONTH)      # cumulative return
          .shift(MONTH)                         # skip most recent month
          .resample('W-FRI').last()             # align to Friday
          .stack()                              # long format
          .to_frame('mom12m'))                 # named DataFrame
```

Motivation

- **Skipping the last month** avoids contamination from short-term reversal
- Isolates **medium-term momentum**: past winners tend to keep winning
- `chmom` refines this by detecting whether momentum is **accelerating or decelerating**

Industry Momentum — Sector-Level Aggregation

`indmom` is a **sector-level** control variable, broadcast to each stock:

Step 1: compute equal-weighted sector average 12-month return

```
indmom = (close.pct_change(12*MONTH).resample('W-FRI').last()  
         .stack().to_frame('close')  
         .join(metadata[['sector']])  
         .groupby(['date', 'sector']).close.mean()  
         .to_frame('indmom').reset_index())
```

Step 2: broadcast sector value to every stock

```
indmom = (returns.stack().to_frame('ret')  
         .join(metadata[['sector']]).reset_index()  
         .merge(indmom)  
         .set_index(['date', 'ticker']).loc[:, ['indmom']])
```

This separates **stock-specific momentum** from **industry-wide trends**, a standard control in cross-sectional asset pricing studies.

Liquidity Metrics

Factor	Definition
turn	3-month average daily volume $\div$ shares outstanding
turn_std	Std. dev. of daily share turnover over 1 month
mvel	$\ln(1 + \text{price-scaled market cap})$
dolvol	$\ln(1 + \text{avg. daily dollar volume, lagged 1 month})$
ill	Amihud illiquidity: $\frac{1}{M} \sum_k \frac{ r_{i,k} }{V_{i,k} \cdot P_{i,k}}$

Amihud (2002) Measure

Captures **price impact**: illiquid stocks move more per unit of traded value.
Higher values $\Rightarrow$ less liquid.

Log Transforms

Dollar volume and market equity use $\ln(1 + x)$ to handle **right-skewed distributions** and stabilise variance.

Liquidity — Sample Code

```
# Turnover: 3-month average volume / shares outstanding
turn = (volume.rolling(3*MONTH).mean()
        .resample('W-FRI').last()
        .div(metadata.sharesoutstanding)
        .stack('ticker').to_frame('turn'))
```

```
# Amihud illiquidity: mean( |daily return| / dollar volume )
dv = close.mul(volume)
ill = (close.pct_change().abs().div(dv)
       .rolling(21).mean()
       .resample('W-FRI').last()
       .stack().to_frame('ill'))
```

All liquidity factors share the same pattern: compute a daily quantity, apply a rolling window, resample to weekly Fridays, and reshape to long format.

Risk Measures — Market Beta

Market proxy: equal-weighted cross-sectional mean of weekly returns.

```
index = (close.resample('W-FRI').last().pct_change()  
         .mean(axis=1).to_frame('x'))
```

Rolling OLS over a 3-year (156-week) window:

```
def get_market_beta(y, x=index):  
    df = x.join(y.to_frame('y')).dropna()  
    model = RollingOLS(endog=df.y,  
                       exog=sm.add_constant(df[['x']]),  
                       window=3*52)  
    return model.fit(params_only=True).params['x']  
  
beta = (returns.dropna(thresh=3*52, axis=1)  
        .apply(get_market_beta).stack().to_frame('beta'))
```

The estimated beta at week t is:

$$\hat{\beta}_{i,t} = \frac{\sum_{k=t-155}^t (x_k - \bar{x})(r_{i,k} - \bar{r}_i)}{\sum_{k=t-155}^t (x_k - \bar{x})^2}$$

Risk Measures — Volatility & Idiosyncratic Risk

retvol — Return Volatility

Standard deviation of daily returns over the prior month:

```
retvol = (close.pct_change()  
         .rolling(21).std()  
         .resample('W-FRI').last()  
         .stack()  
         .to_frame('retvol'))
```

Risk Measures — Volatility & Idiosyncratic Risk

idiovol — Idiosyncratic Volatility

Residual std. dev. from a market-model regression over 3 years:

```
def get_ols_residuals(y, x=index):  
    df = x.join(y.to_frame('y')).dropna()  
    model = sm.OLS(df.y,  
                   sm.add_constant(df[['x']]))  
    return model.fit().resid.std()
```

Applied via `rolling(3*52).apply(...)`.

Risk Measures — Volatility & Idiosyncratic Risk

betasq — Beta Squared

Simply $\hat{\beta}_{i,t}^2$. Captures **non-linear** exposure to market risk; relevant because expected returns may depend on higher moments of factor loadings.

Section 3.3

Storage & Output

HDF5 Database Layout

All outputs are stored in a single HDF5 file under hierarchical keys:

```
with pd.HDFStore(results_path / 'autoencoder_reloaded.h5') as store:  
    store.put('close', close)  
    store.put('volume', volume)  
    store.put('returns', returns)  
    store.put('metadata', metadata)
```

HDF5 Database Layout

HDF5 Key	Content	Index
close / volume	Daily prices / volume (wide)	date $\times$ ticker
returns	Weekly returns (wide)	date $\times$ ticker
metadata	Sector, mktcap, shares	ticker
factor/mom1m	Short-term reversal	(date, ticker)
factor/mom12m	Momentum	(date, ticker)
factor/chmom	Momentum change	(date, ticker)
factor/indmom	Industry momentum	(date, ticker)
factor/maxret	Recent max return	(date, ticker)
factor/mom36m	Long-term reversal	(date, ticker)
factor/turn ... factor/ill	Liquidity metrics	(date, ticker)
factor/beta ... factor/idiovol	Risk measures	(date, ticker)

Section 3.4

Summary of Part 3

Summary & Next Steps

What This Notebook Produces

A structured HDF5 file containing **weekly returns** and **14 firm-level characteristics** for thousands of U.S. equities, all aligned on a consistent (date, ticker) panel at Friday frequency.

Key Design Principles

- Universe alignment: prices $\cap$ metadata
- Consistent weekly (Friday) sampling
- Rolling windows for time-varying features
- Economic motivation for every factor

Downstream Usage (Next Chapter)

- **Conditioning variables** for the autoencoder's encoder network
- **Cross-sectional normalisation** (rank transforms)
- **Train / validation / test** split
- **Latent factor extraction** and SDF estimation